

Objetivo

- Desenvolver uma aplicação funcional em Python para gerenciamento de pets.
- Utilizar o Oracle como banco de dados persistente.
- Implementar CREATE, READ, UPDATE e DELETE.
- Organizar o programa por meio de funções, sem classes.
- Tratar erros de entrada e erros do Oracle sem encerrar indevidamente o sistema.
- Gerar dinamicamente um relatório HTML com os dados armazenados.

Orientações:

- A atividade é em grupo com no máximo cinco pessoas.
- É obrigatório utilizar Python e a biblioteca python-oracledb.
- É proibido utilizar classes, objetos ou qualquer estrutura de Orientação a Objetos.
- O programa deve ser organizado em funções.
- As operações de banco devem ser realizadas pelo Python.
- Os comandos SQL devem utilizar bind variables; não concatene valores digitados pelo usuário diretamente no SQL.
- INSERT, UPDATE e DELETE devem utilizar commit() quando concluídos com sucesso.
- Em caso de erro em uma operação de alteração de dados, deve ser utilizado rollback().
- O programa deve continuar funcionando após erros previsíveis.
- Cursor e conexão devem ser fechados adequadamente.
- O sistema deve possuir um menu principal.

Banco de dados

Crie a tabela abaixo no Oracle:

```
CREATE TABLE VIDAPET (  
    ID_PET NUMBER PRIMARY KEY,  
    NOME_PET VARCHAR2(100) NOT NULL,  
    TIPO_PET VARCHAR2(30) NOT NULL,  
    RACA VARCHAR2(60),  
    IDADE NUMBER NOT NULL,  
    PESO NUMBER(6,2),  
    NOME_TUTOR VARCHAR2(100) NOT NULL,  
    TELEFONE VARCHAR2(20),  
    DATA_CADASTRO DATE DEFAULT SYSDATE  
);
```

Regras dos dados:

- ID_PET é a chave primária e não pode se repetir.
- IDADE deve ser inteira e maior ou igual a zero.
- PESO, quando informado, deve ser maior que zero.
- DATA_CADASTRO deve ser preenchida automaticamente pelo Oracle com DEFAULT SYSDATE.
- Não é necessário criar sequence para esta atividade.

Conexão com o Oracle

Utilize a estrutura trabalhada em aula, substituindo usuário e senha pelas credenciais fornecidas ao estudante:

```
USUARIO = "SEU_USUARIO"
SENHA = "SUA_SENHA"
HOST = "oracle.fiap.com.br"
PORTA = 1521
SERVICE_NAME = "ORCL"

dsn = oracledb.makedsn(
    HOST,
    PORTA,
    service_name=SERVICE_NAME
)
```

Na entrega, não exponha sua senha em prints ou repositórios públicos.

Menu principal

O programa deverá apresentar, no mínimo:

- 1 - Cadastrar pet
- 2 - Consultar todos os pets
- 3 - Consultar pet por ID
- 4 - Alterar pet
- 5 - Excluir pet
- 6 - Gerar relatório HTML
- 0 - Sair

Após cada operação, o sistema deve retornar ao menu, permitindo novas operações sem reiniciar o programa.

CREATE — Cadastro

A opção 1 deverá:

- Solicitar ID, nome, tipo, raça, idade, peso, nome do tutor e telefone.
- Validar os campos obrigatórios.
- Impedir idade negativa.
- Impedir peso menor ou igual a zero quando informado.
- Executar INSERT no Oracle usando bind variables.
- Executar commit() somente após o INSERT ser concluído com sucesso.
- Informar claramente o resultado da operação.
- Tratar, sem encerrar o programa, situações como ID duplicado, dado inválido ou erro do Oracle.

READ — Consultas

Consultar todos

- Executar SELECT no Oracle.
- Exibir os registros de forma organizada no terminal.
- Informar quando não houver registros.
- Os dados apresentados devem vir do Oracle.

Consultar por ID

- Solicitar o ID ao usuário.
- Executar SELECT com WHERE ID_PET = :id_pet.
- Exibir todos os dados encontrados.
- Informar quando o ID não existir.
- Tratar entrada inválida.

UPDATE — Alteração

- Solicitar o ID do pet.
- Verificar se o registro existe antes de alterar.
- Permitir alterar nome, tipo, raça, idade, peso, nome do tutor e telefone.
- Executar UPDATE com bind variables.
- Utilizar commit() em caso de sucesso.
- Utilizar rollback() em caso de erro.
- Apresentar mensagem clara ao usuário.

DELETE — Exclusão

- Solicitar o ID.
- Verificar se o pet existe.
- Exibir os dados básicos do registro antes da exclusão.
- Solicitar confirmação do usuário (S/N).
- Excluir somente após confirmação.
- Executar commit() após sucesso.
- Executar rollback() se ocorrer erro.
- Informar o resultado da operação.

Exportação para HTML

A opção 6 deverá gerar obrigatoriamente o arquivo relatorio_vidapet.html. O arquivo deve ser construído pelo Python a partir de um SELECT realizado no Oracle. Não é permitido copiar os dados manualmente para o HTML.

- O documento deve possuir <!DOCTYPE html>, html, head e body.
- Utilizar charset UTF-8.
- Possuir título e cabeçalho identificando a VidaPet.
- Criar uma tabela HTML com os registros.

- Exibir: ID, nome, tipo, raça, idade, peso, tutor, telefone e data de cadastro.
- Utilizar CSS para melhorar a apresentação.
- Apresentar uma mensagem adequada quando não houver registros.
- Exibir a data/hora de geração do relatório, obtida pelo Python.
- O HTML deve refletir os dados existentes no Oracle no momento da geração.

Organização obrigatória por funções

A solução deve ser procedural. Recomenda-se utilizar funções semelhantes às seguintes:

- `conectar_banco()` — abre a conexão.
- `cadastrar_pet()` — executa CREATE/INSERT.
- `consultar_pets()` — consulta todos.
- `consultar_pet_por_id()` — consulta um registro.
- `alterar_pet()` — executa UPDATE.
- `excluir_pet()` — executa DELETE.
- `gerar_html()` — consulta o Oracle e gera o relatório.
- `menu()` — controla o sistema.

Você pode criar funções auxiliares para validações, confirmação de exclusão, formatação e tratamento de erros.

Testes mínimos obrigatórios

Teste	Procedimento	Resultado esperado
01	Cadastrar pelo menos 3 pets.	Os registros ficam persistidos no Oracle.
02	Tentar cadastrar ID já existente.	O erro é tratado e o sistema continua.
03	Consultar todos.	Os registros são exibidos.
04	Consultar ID existente.	O pet correto é apresentado.
05	Consultar ID inexistente.	Sistema informa que não encontrou.
06	Alterar um registro.	Novos dados ficam persistidos.
07	Excluir confirmando S.	Registro é removido.
08	Excluir respondendo N.	Registro permanece.
09	Gerar HTML.	Arquivo é criado pelo Python.
10	Abrir HTML no navegador.	Tabela mostra os dados atuais.
11	Fornecer entrada inválida.	Programa trata o erro sem encerrar.

Desafios de qualidade

- O sistema deve continuar funcionando após uma tentativa de cadastro inválida.
- O sistema deve continuar funcionando após erro de alteração ou exclusão.
- Operações de alteração de dados devem manter a consistência da transação.
- O relatório deve ser regenerado com os dados atuais do Oracle.
- As mensagens do sistema devem ser claras e coerentes.
- O código deve evitar repetição desnecessária.

Entrega

- Arquivo Python, por exemplo: `atividade_vidapet.py`.

- Arquivo gerado pelo programa: relatorio_vidapet.html.
- Evidências de testes, conforme orientação do professor.

As evidências devem demonstrar, preferencialmente, cadastro, consulta, alteração, exclusão e HTML aberto no navegador.

Critérios de avaliação — 10,0 pontos

Critério	Descrição	Valor
Conexão com Oracle	Conexão funcional e gerenciamento adequado de recursos.	1,0
CREATE	Cadastro, validações, bind variables, commit e tratamento de erros.	1,5
READ	Consulta geral e por ID, com resultados corretos.	1,5
UPDATE	Verificação, alteração, commit/rollback e mensagens.	1,5
DELETE	Confirmação, exclusão, commit/rollback e tratamento.	1,5
HTML	Geração dinâmica, tabela completa, dados atuais e CSS.	1,5
Tratamento de erros	Entradas inválidas e erros previsíveis tratados sem encerramento.	0,5
Organização procedural	Uso de funções, clareza, indentação e ausência de classes.	0,5
Testes e entrega	Demonstração das funcionalidades e arquivos solicitados.	0,5